

## 53. Maximum Subarray Medium

Acceptance: 52.8% | [LeetCode](#)

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

### Example 1:

```
**Input:** nums = [-2,1,-3,4,-1,2,1,-5,4]
**Output:** 6
**Explanation:** The subarray [4,-1,2,1] has the
largest sum 6.
```

### Example 2:

```
**Input:** nums = [1]
**Output:** 1
**Explanation:** The subarray [1] has the largest sum
1.
```

### Example 3:

```
**Input:** nums = [5,4,-1,7,8]
**Output:** 23
**Explanation:** The subarray [5,4,-1,7,8] has the
largest sum 23.
```

### Constraints:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`

**Follow up:** If you have figured out the  $O(n)$  solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

## Claude Opus Solution

## Maximum Subarray — Kadane's Algorithm

### 1. Intuition

The key insight is a simple decision made at **every element**:

"As I extend my subarray to include the current element, should I **continue** the previous subarray, or **start fresh** from the current element?"

The answer hinges on one question: **Is the running sum so far helping or hurting me?**

- If the previous subarray's sum is **positive**, it boosts the current element → keep it.
- If the previous subarray's sum is **negative**, it drags down the current element → discard it and start over.

This greedy/DP observation is **Kadane's Algorithm**. We never need to look at all possible subarrays (which would be  $O(n^2)$ ); we only track the best subarray *ending at each position*.

### 2. Approach (Pseudocode)

Define:

```
current_sum = best subarray sum ENDING at index i
max_sum      = best subarray sum seen ANYWHERE so far
```

Recurrence:

```
current_sum(i) = max(nums[i], current_sum(i-1) + nums[i])
               └start fresh┘ └── extend previous ─┘
```

Algorithm:

1. Initialize `current_sum = max_sum = nums[0]`
2. For each num from index 1 to end:
  - a. `current_sum = max(num, current_sum + num)`
  - b. `max_sum = max(max_sum, current_sum)`
3. Return `max_sum`

### 3. Python Solution

```
from typing import List
```

```
class Solution:
```

```
    def maxSubArray(self, nums: List[int]) -> int:
        # `current_sum`: the maximum sum of a subarray that ENDS at the
```

```

#           current index. This is our running DP state.
# `max_sum`:   the global best answer found so far.
# Initialize both to the first element (subarray must be non-empty).
current_sum = max_sum = nums[0]

# Process each element starting from the second one.
for num in nums[1:]:
    # Decision: either start a brand-new subarray at `num`,
    # or extend the previous best subarray by adding `num`.
    # If `current_sum` is negative, `num` alone is better,
    # so we effectively "reset" the subarray.
    current_sum = max(num, current_sum + num)

    # Update the global maximum if the new ending-here sum is larger.
    max_sum = max(max_sum, current_sum)

return max_sum

```

**Note on the reference solution:** It mutates `nums` in place to store `current_sum`, then takes a final `max(nums)`. That's elegant but modifies the input. The version above is cleaner and keeps `nums` untouched.

## 4. Complexity Analysis

| Metric | Complexity | Explanation                                                                    |
|--------|------------|--------------------------------------------------------------------------------|
| Time   | $O(n)$     | Single pass through the array; constant work per element.                      |
| Space  | $O(1)$     | Only two scalar variables ( <code>current_sum</code> , <code>max_sum</code> ). |

(The reference solution is also  $O(n)$  time, but its in-place mutation doesn't allocate extra space either, so  $O(1)$ .)

## 5. Alternative Approaches

### A) Divide and Conquer — $O(n \log n)$ (the follow-up)

Split the array in half. The maximum subarray is one of three cases: 1. Entirely in the **left** half. 2. Entirely in the **right** half. 3. **Crossing** the midpoint.

We recurse on left and right, compute the best crossing sum linearly, and take the max.

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def helper(lo, hi):
            if lo == hi:
                # base case: single element
                return nums[lo]

            mid = (lo + hi) // 2

            # Best subarray fully in left / right halves (recursive).
            left = helper(lo, mid)
            right = helper(mid + 1, hi)

            # Best subarray crossing the midpoint:
            # Expand leftward from mid, and rightward from mid+1.
            cur, left_max = 0, float('-inf')
            for i in range(mid, lo - 1, -1): # walk left
                cur += nums[i]
                left_max = max(left_max, cur)

            cur, right_max = 0, float('-inf')
            for i in range(mid + 1, hi + 1): # walk right
                cur += nums[i]
                right_max = max(right_max, cur)

            cross = left_max + right_max
            return max(left, right, cross)

        return helper(0, len(nums) - 1)

```

• **Time:**  $O(n \log n)$  (recurrence  $T(n) = 2T(n/2) + O(n)$ ).

- **Space:**  $O(\log n)$  recursion stack.
- More subtle, slower than Kadane — useful as an algorithmic exercise.

## B) Prefix Sum + Min Tracking — $O(n)$

The sum of subarray  $[i..j] = \text{prefix}[j] - \text{prefix}[i-1]$ . To maximize this for a fixed  $j$ , minimize  $\text{prefix}[i-1]$ :

```
def maxSubArray(self, nums):
    prefix = 0
    min_prefix = 0      # smallest prefix sum seen before current index
    best = float('-inf')
    for num in nums:
        prefix += num
        best = max(best, prefix - min_prefix) # best subarray ending here
        min_prefix = min(min_prefix, prefix) # update minimum for future
    return best
```

- **Time:**  $O(n)$ , **Space:**  $O(1)$ . Equivalent in power to Kadane.

## ✓ Recommendation

Use **Kadane's Algorithm** — it's optimal ( $O(n)$  time,  $O(1)$  space), short, and the canonical solution. Keep the divide-and-conquer version in your toolkit for interviews that explicitly ask for it.

## Codex Solution

### 1. Intuition

The key insight is:

At every index, we only need to know the best subarray sum that **must end at the current index**.

For each number `nums[i]`, we have two choices:

1. Start a new subarray at `nums[i]`
2. Extend the previous subarray by adding `nums[i]`

So the best subarray ending at index  $i$  is:

```
current_sum = max(nums[i], current_sum + nums[i])
```

Then, while scanning the array, we keep track of the largest `current_sum` we have ever seen.

This algorithm is known as **Kadane's Algorithm**.

### 2. Approach

#### Pseudocode

```
Initialize current_sum = nums[0]
Initialize best_sum = nums[0]

For each index i from 1 to len(nums) - 1:
    current_sum = max(nums[i], current_sum + nums[i])
    best_sum = max(best_sum, current_sum)

Return best_sum
```

#### Why this works

At each position:

```
current_sum
```

means:

```
Maximum subarray sum ending exactly at the current index
```

If the previous sum is negative, it hurts the future subarray, so we discard it and start fresh.

For example:

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

When we reach 4, it is better to start fresh from 4 rather than extend the previous negative sum.

The best subarray becomes:

```
[4, -1, 2, 1]
```

with sum:

```
6
```

---

### 3. Python Solution

```
from typing import List

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # current_sum represents the maximum subarray sum
        # that ends exactly at the current index.
        current_sum = nums[0]

        # best_sum represents the maximum subarray sum
        # found anywhere in the array so far.
        best_sum = nums[0]

        # Start from the second element because we initialized
        # current_sum and best_sum using nums[0].
        for i in range(1, len(nums)):
            num = nums[i]

            # At each index, we decide:
            #
            # 1. Start a new subarray at nums[i]
            # 2. Extend the previous subarray by adding nums[i]
            #
            # If current_sum is negative, starting fresh is better.
            current_sum = max(num, current_sum + num)

            # Update the global maximum if the best subarray
            # ending at this index is better than anything seen before.
            best_sum = max(best_sum, current_sum)

        return best_sum
```

---

### 4. Complexity Analysis

Let  $n$  be the length of `nums`.

#### Time Complexity

```
 $O(n)$ 
```

We scan the array once.

Each element is processed in constant time.

#### Space Complexity

```
 $O(1)$ 
```

We only use two variables:

```
current_sum
best_sum
```

No extra array is needed.

---

## 5. Alternative Approaches

### Approach 1: Modify the Input Array as DP

This is close to the given reference solution.

```
from typing import List

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # nums[i] will store the maximum subarray sum ending at index i.
        for i in range(1, len(nums)):
            nums[i] = max(nums[i], nums[i - 1] + nums[i])

        return max(nums)
```

#### Complexity

Time:  $O(n)$   
Space:  $O(1)$

However, this modifies the input array, which may not always be desirable.

---

### Approach 2: Dynamic Programming Array

Instead of modifying `nums`, we can create a separate DP array.

```
from typing import List

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        n = len(nums)

        # dp[i] stores the maximum subarray sum ending at index i.
        dp = [0] * n
        dp[0] = nums[0]

        best_sum = dp[0]

        for i in range(1, n):
            dp[i] = max(nums[i], dp[i - 1] + nums[i])
            best_sum = max(best_sum, dp[i])

        return best_sum
```

#### Complexity

Time:  $O(n)$   
Space:  $O(n)$

This is easier to understand but less space-efficient than Kadane's Algorithm.

---

### Approach 3: Divide and Conquer

The follow-up asks for a divide and conquer solution.

For any range `[left, right]`, the maximum subarray can be one of three cases:

1. Entirely in the left half
2. Entirely in the right half
3. Crosses the middle

For each segment, we can compute:

```
total_sum
max_prefix_sum
```

```
max_suffix_sum  
max_subarray_sum
```

Then merge results from the left and right halves.

### Complexity

```
Time:  $O(n)$   
Space:  $O(\log n)$ 
```

The space is from recursion depth.

Kadane's Algorithm is simpler and usually preferred for this problem.